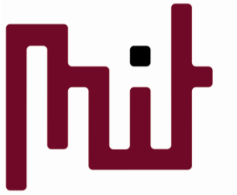
Beágyazottszoftver-fejlesztés
2026. 09. 10.

Bevezető



Mesterséges Intelligencia és
Rendszertervezés
Tanszék

© BME-MIT

Tudnivalók

- **Beágyazottszoftverfejlesztés**
  - Tárgykód: BMEVIMIAC17
  - Órák:
    - Előadás: minden csütörtök 8:30-10:00
    - Gyakorlat: kedd 12:15-13:45
      - Programozási feladatok közösen megoldva (Simplicity Studio 5)
      - Tanszéki termekben: IE.224, IE.225



  - Oktatók:
    - Orosz György (tárgyfelelős), Naszály Gábor, Scherer Balázs, Varga Balázs
    - Email: orosz@mit.bme.hu



  - Követelmények:
    - Zárthelyi dolgozat: 2026. nov. 13., péntek, 14:00-16:00
      - pótZH: 2026. dec. 01., kedd, 18:00 – 20:00
    - Otthoni feladat: egyszerű feladat mikrokontrolleres kártyára

Beágyazott rendszerek

- **Lehetséges definíciók**
  - Azok a számítógépes alkalmazói rendszerek, amelyek:
    - autonóm működésűek,
    - intenzív információs kapcsolatban vannak a fizikai/technológiai környezetükkel.
  - Olyan egység, ami vezérel vagy felügyel egy gépet, berendezést vagy ipari folyamatot.
  - Az a számítógép, aminek nincsen billentyűzete ☺. Tehát minden olyan processzoros vagy digitális egység, amely nem személyi számítógép.
- **Lényegében a hagyományos mikroprocesszoros rendszerek is beágyazott rendszereknek tekinthetők.**

Beágyazott rendszerek: példák

- **Példák:**
  - Szórakoztató elektronika: zenelejátszó, TV, karóra, vezeték nélküli fejhallgató, fényképező, monitor, vezeték nélküli egér/billentyűzet
  - Használati eszközök: Mobiltelefon, GPS, számológép
  - Háztartás: mosógép, mikrohullámú sütő, hűtő…
  - Épületautomatizálás: liftvezérlő, riasztórendszer, fűtésvezérlés, távoli lakásfelügyelet
  - Gépjármű-elektronika: motorvezérlő, ABS, szervokormány, tükörmozgató, fedélzeti számítógép, szórakoztatóelektronikai eszközök, tolatóradar, sebességváltó-vezérlő
  - Ipari robotok, motorvezérlők, intelligens tápegységek
  - Jegykiadó automata, bankjegy-automata, elektronikus információs központok
  - Egészségügyi műszerek: vérnyomásmérő, komplett diagnosztikai eszközök
  - Mérőműszerek: mai modern műszerek szoftveres vezérlésűek
  - Telekommunikációs eszközök: modem, router, switch

Beágyazott rendszerek fejlesztői

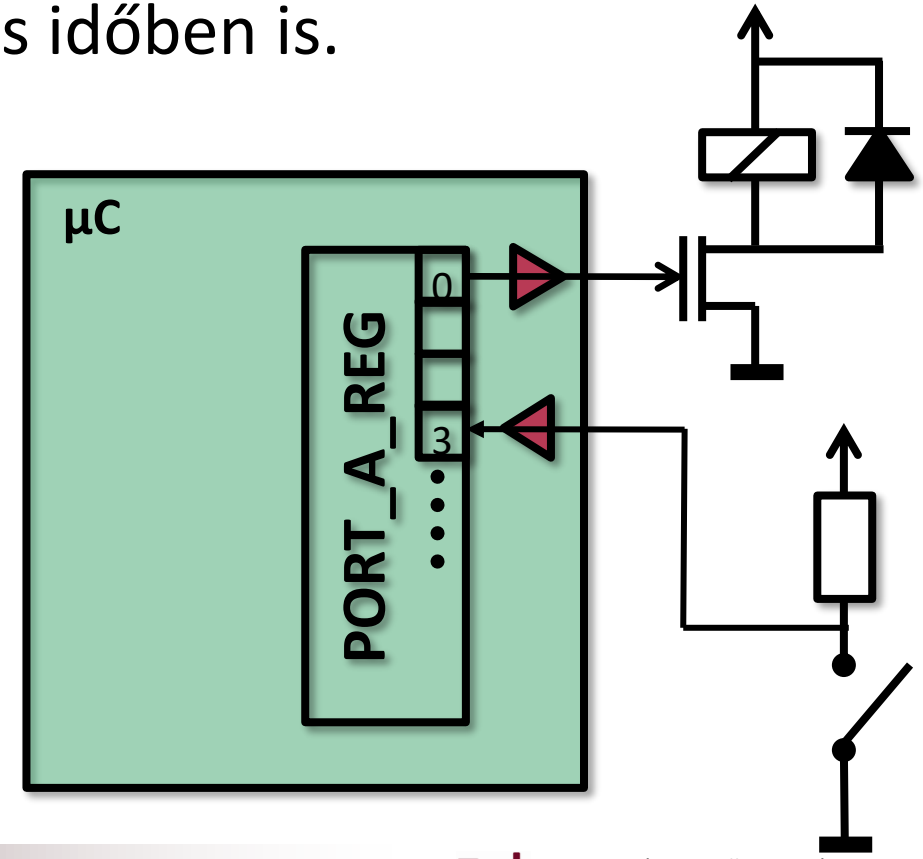
- **Miért jó beágyazott rendszerekkel foglalkozni?**
  - Fejlesztés során a fizikai és szoftveres világ határán mozgunk: a megírt szoftver képes közvetlenül információkat nyerni a fizikai környezetből és képes beavatkozni a fizikai környezetbe.
  - Kapcsolatba kerülünk az áramköri szinttől kezdve a szoftverfejlesztésen át, a PC-kel és magasabb szintű informatikai rendszerekkel.
  - Folyamatosan fejlődő iparág, a mai napig rengetegen helyezkednek el a területen, folyamatosan új szakembereket kíván.



Példa: közvetlen kapcsolat a környezettel

- Mit értünk az alatt, hogy közvetlen kapcsolatban vannak a fizikai környezetükkel?
  - Alacsony szinten képesek a környezet jeleit érzékelni, arra reagálni, akár valós időben is.

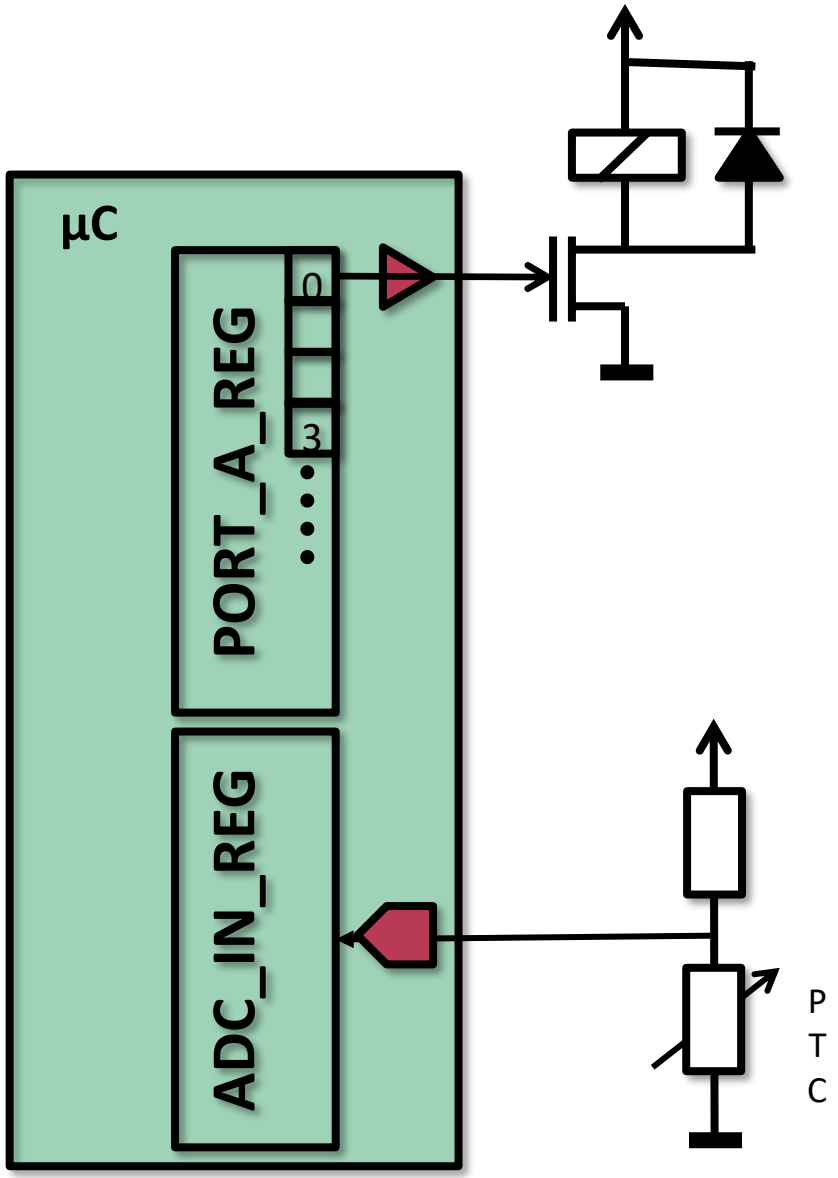
```
#define RELAY_ON  (0)
#define DOOR_BIT  (3)
.
.
.
// PORT_A_REG: memóriába ágyazott I/O regiszter
door_is_open = PORT_A_REG & (1 << DOOR_BIT);
if (door_is_open){
    PORT_A_REG |= 1 << RELAY_ON;
}
.
.
.
```



Példa: közvetlen kapcsolat a környezettel

- Példa2: fűtésszabályozás:
  - Egy PTC (Positive Temperature Coefficient) hőmérsékletfüggő ellenállással mérjük a hőmérsékletet
  - Analóg-digitális átalakítóval (ADC) digitalizáljuk a hőmérsékletfüggő feszültséget
  - A hőmérséklet függvényében kapcsoljuk a fűtést kapcsoló relét.

```
#define FUTES_ON_BIT          (0)
#define HOMERSEKLET_LIMIT  (25) //alatta fűtés be
#define HOMERSEKLET_HISZT (2)   //hiszterézis
#define SKALA_TENYEZO         (0.145)
.
.
// PORT_A_REG: memóriába ágyazott I/O regiszter
// ADC_IN_REG: memóriába ágyazott ADC eredmény regisztere
homerseklet = ADC_IN_REG*SKALA_TENYEZO;
if (homerseklet < HOMERSEKLET_LIMIT){
     PORT_A_REG |= 1 << FUTES_ON_BIT;
} else
if  (homerseklet > (HOMERSEKLET_LIMIT+HOMERSEKLET_HISZT)){
     PORT_A_REG &= ~(1 << FUTES_ON_BIT);
}
.
.
```



Beágyazott rendszerek fejlődése

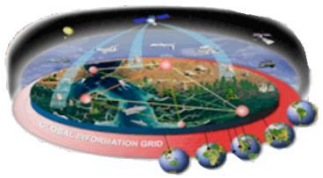
- **Fejlődéstörténet, mérföldkövek röviden**
  - 1960-as évek: első említett beágyazott rendszer az Apollo programban használt vezérlők.
  - 1970-es évek: nagy tömegben gyártott és elterjedt mikroprocesszorok (pl. 8086), személyi számítógépek.
  - 1980-as évek: integrált perifériakészlettel ellátott *mikrokontrollerek*.
  - 1990-es évek: mobil eszközök megjelenése, beágyazott rendszerek elterjedése a háztartásokban. SoC rendszerek fejlődése.
  - 2000-es évek: beágyazott rendszerek egyre inkább beépülnek az átlagemberek mindennapjaiba
    - Ambiens rendszerek (ambient: környezeti, körülvevő)
  - 2010-es évek: beágyazott rendszerek összekapcsolása komplex rendszerekké:
    - Internet of Things („beágyazott eszközök hálózata”)
    - Cyber Physical Systems (beágyazott rendszerek és magas szintű intelligencia és adatbázisok integrációja)







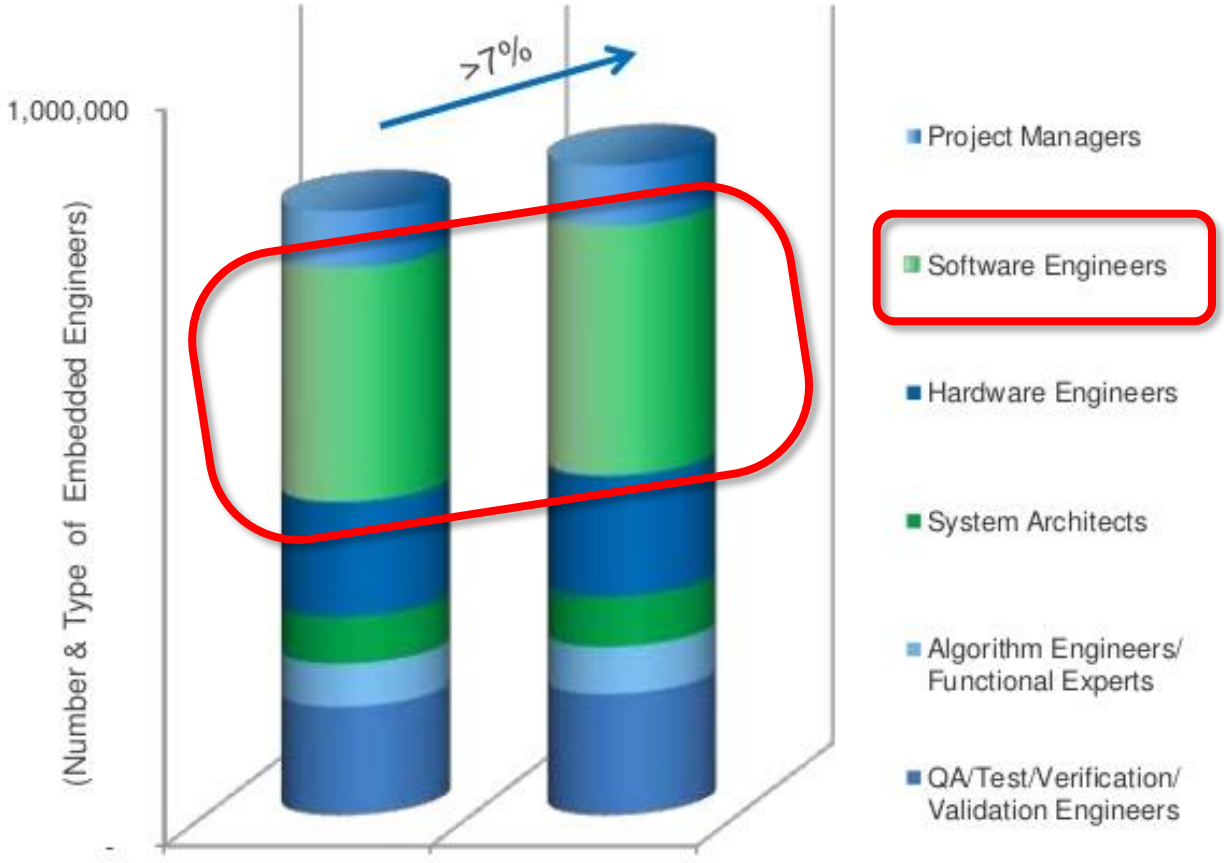


Beágyazott rendszerek fejlesztése

- **Fejlesztési feladatok**
  - Hardverfejlesztés
  - Szoftverfejlesztés
  - Tesztelés
  - Szoros együttműködés a fejlesztés fázisok között (általában céleszközök)
- **Hardverfejlesztés**
  - Áramkör tervezése, megvalósítása, élesztése
  - Áramkör testreszabása a fejlesztési tapasztalatok alapján
  - Habár folyamatos felügyeletet követel, de a hardver a rendszer egyik leglassabban változó komponense
  - Egyre több multifunkciós eszköz létezik, amire „csak” szoftvert kell írni
- **Szoftverfejlesztés**
  - Szoftveres rendszerterv készítése
  - Alacsony szintű és magas szintű szoftverkomponensek fejlesztése
  - Folyamatos fejlődés, dinamikusabban változik, mint a hardver
  - A fejlesztők nagy része szoftveres területen dolgozik
- **A tantárgyon belül a beágyazott szoftverfejlesztésre koncentrálunk**

Mérnöki feladatok

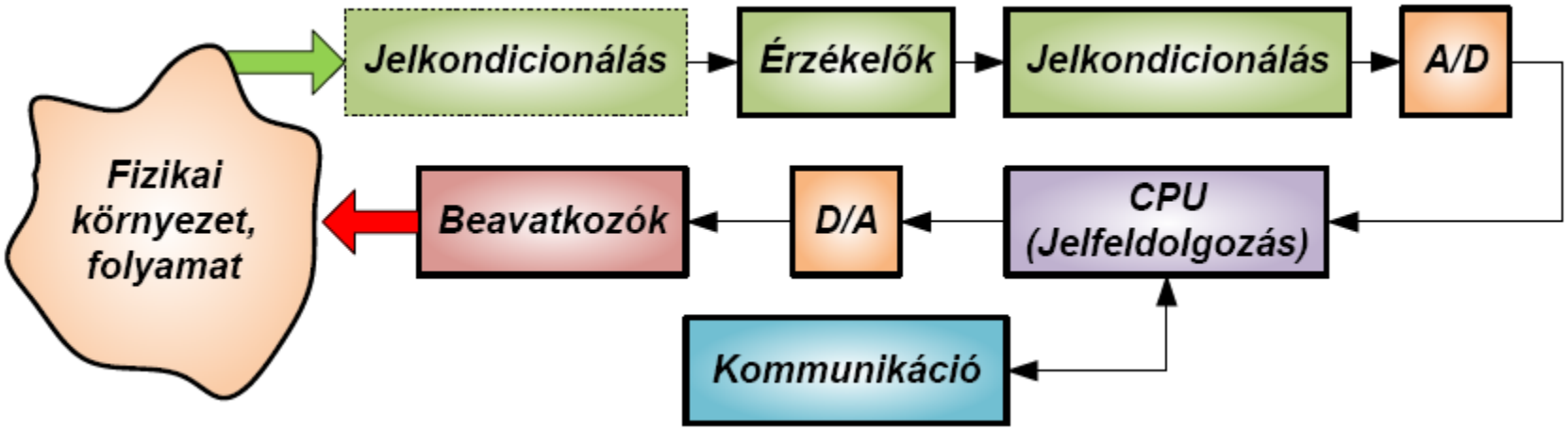


Beágyazott szoftver sajátosságai

- Hardverközeli programozás
- Nem elég a funkcionalitást megírni, a programot (akár rendszerszinten akár forráskód szintjén) tudatosan kell fejleszteni.
  - Figyelembe kell venni az eszköz sajátosságait
  - Általános szoftvereket fejlesztünk, de nem mindenhol tekinthetünk el a használt platformtól
- Takarékoskodni kell az erőforrásokkal:
  - Memória/Adatmennyiség
  - Processzoridő
    - Algoritmusok komplexitása
  - Áramfelvétel
- Futási idő sokszor kritikus (valós idejű rendszerek)
- A kód mögé kell látni, tudni kell, hogy milyen erőforrásokat használ a megírt kód, mennyire takarékos

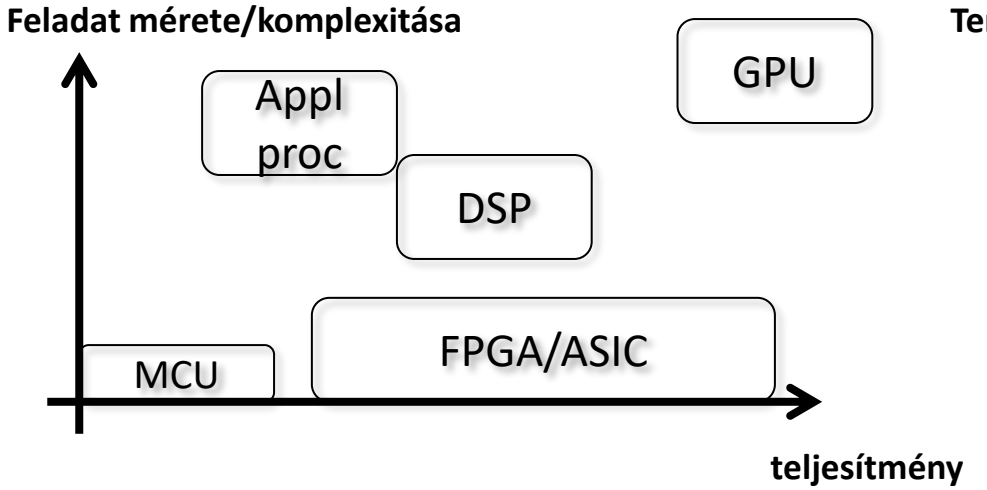
Beágyazott rendszerek felépítése

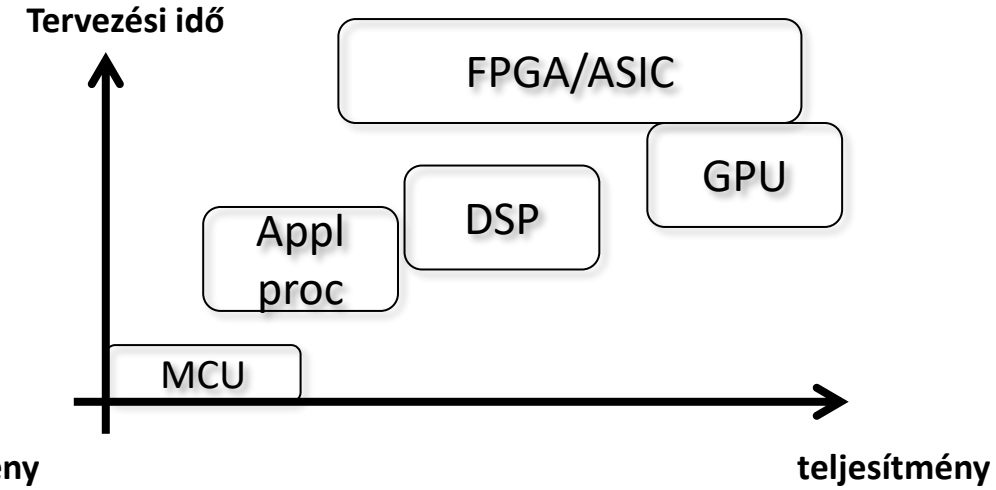
- **Beágyazott rendszerek főbb komponensei:**
  - Csatlakozás a fizikai környezethez (bemenet):
    - Érzékelő/Jelátalakító: hőmérséklet, fény, pozíció, erő, áram
    - Jelkondicionáló
    - Beviteli eszközök: billengyűzet, nyomógomb, kapcsoló
  - Feldolgozó egység
  - Kommunikáció
  - Beavatkozó



Feldolgozó egység

- **Alapvető feldolgozó egységek**
  - Mikroprocesszor
  - Mikrokontroller
  - FPGA (Field Programmable Gate Array)
  - DSP (Digital Signal Processor)
  - GPU (Graphics Processing Unit)
  - ASIC (Application-specific Integrated Circuit)





Mikrokontroller

- Mikrokontroller = mikroprocesszor + integrált perifériák
- Perifériák:
  - Memória (adat és programmemória)
  - Időzítők
  - Kommunikációs egységek (UART, SPI, I2C, CAN, USB, Ethernet)
  - ADC és DAC
  - GPIO (General Purpose Input Output)
  - Energiamenedzsment
  - Debug Interfész
- Tipikus órajeltartomány: 1 MHz … 100 MHz+
- Választási szempontok:
  - Hozzáférhetőség
  - Feladathoz illeszkedő komplexitás, perifériakészlet (pl. autóipar, videó, titkosítás)
  - Ár (nem csak a chip, hanem pl. fejlesztőkörnyezet, debugger)
  - Tapasztalatok
  - Támogatottság (technikai support, fórumok, függvénykönyvtárak, fejlesztői környezet, mintapéldák, debug lehetőségek)
  - Tokozás

Mikrokontroller mintapéldák



ATTiny25
340 Ft (v.ö.: 1db inverter: 50..150Ft)
8 bites architektúra
10 MHz
2 kB prog MEM
128 Byte RAM
2 Timer, 2 PWM
10 bit ADC 4 csat.
6 GPIO (spec funkc)
SPI



ATmega128
2000…5400 Ft
8 bites architektúra
16 MHz
128 kB prog MEM
4 kByte RAM
4kB EEPROM
8 bit HW szorzó
5 Timer, 8 PWM
10 bit ADC 8 csat
53 GPIO (spec funkc)
SPI, UART, I2C



EFM32GG995 (Gecko)
8000 Ft
32 bites architektúra
48 MHz
1024 kB prog MEM
128 kByte RAM
6 Timer
12 bit ADC 8 csatorna
12 bit DAC 2 csatorna
93 GPIO (spec funkc)
3*SPI, 2*UART, 2*I2C, USB
OPA
Sensor Interface
DMA
HW titkosítás
LCD vezérlő



Balckfin BF537
12000 Ft
16/32 bites architektúra
600 MHz
1db 16 bites MAC
2 db 40 bites ALU
4db 8 bites ALU (video)
Párhuzamos műveletvégzés
132 kB prog/adat RAM
Párhuzamos adatmozgatás
HW ciklusszervezés
Cirkuláris buffer
8 Timer
48 GPIO (spec funkc)
SPI, 2*UART, 2*I2C, Ethernet, CAN, I2S
DMA

Mikrokontroller mintapéldák

- Mikrokontrollerek és alkalmazásprocesszorok

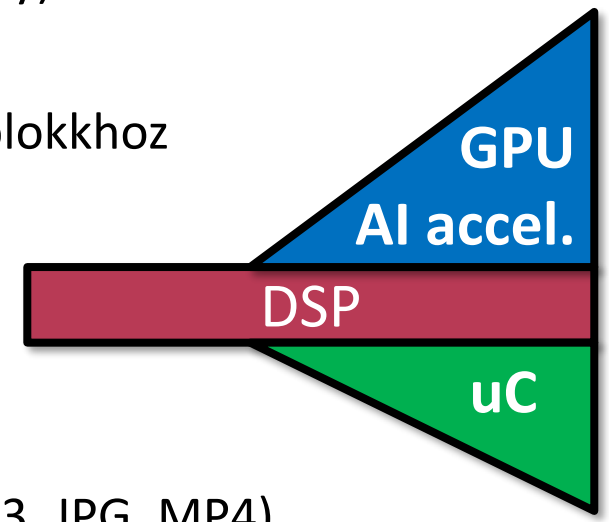
	Mikrokontroller (pl. Cortex-M3)	**Alkalmazásprocesszor**
alkalmazás	Kisteljesítményű, sokszor valós idejű alkalmazások (pl. szabályzás/vezérlés)	Nagyteljesítményű alkalmazások (mobiltelefon, tablet, multimédia)
Futtatási környezet	Általában C vagy C++	Valamilyen operációs rendszer (Linux, RTOS, Android)
Memóriakezelés	Egyszerű memóriakezelés	MMU: Memory management unit (OS által megkívánt, virtuális memóriakezelés)
Perifériák	USART, SPI, ADC, Timer…	Ethernet, videó dekóder
Számítási teljesítmény	Alacsony-közepes	Magas, lebegőpontos, SIMD, DSP utasításkészlet, akár 64 bites architektúra, párhuzamosítás



a tárgyon belül ezzel a szegmenssel foglalkozunk

Jelfeldolgozó processzorok (DSP-k)

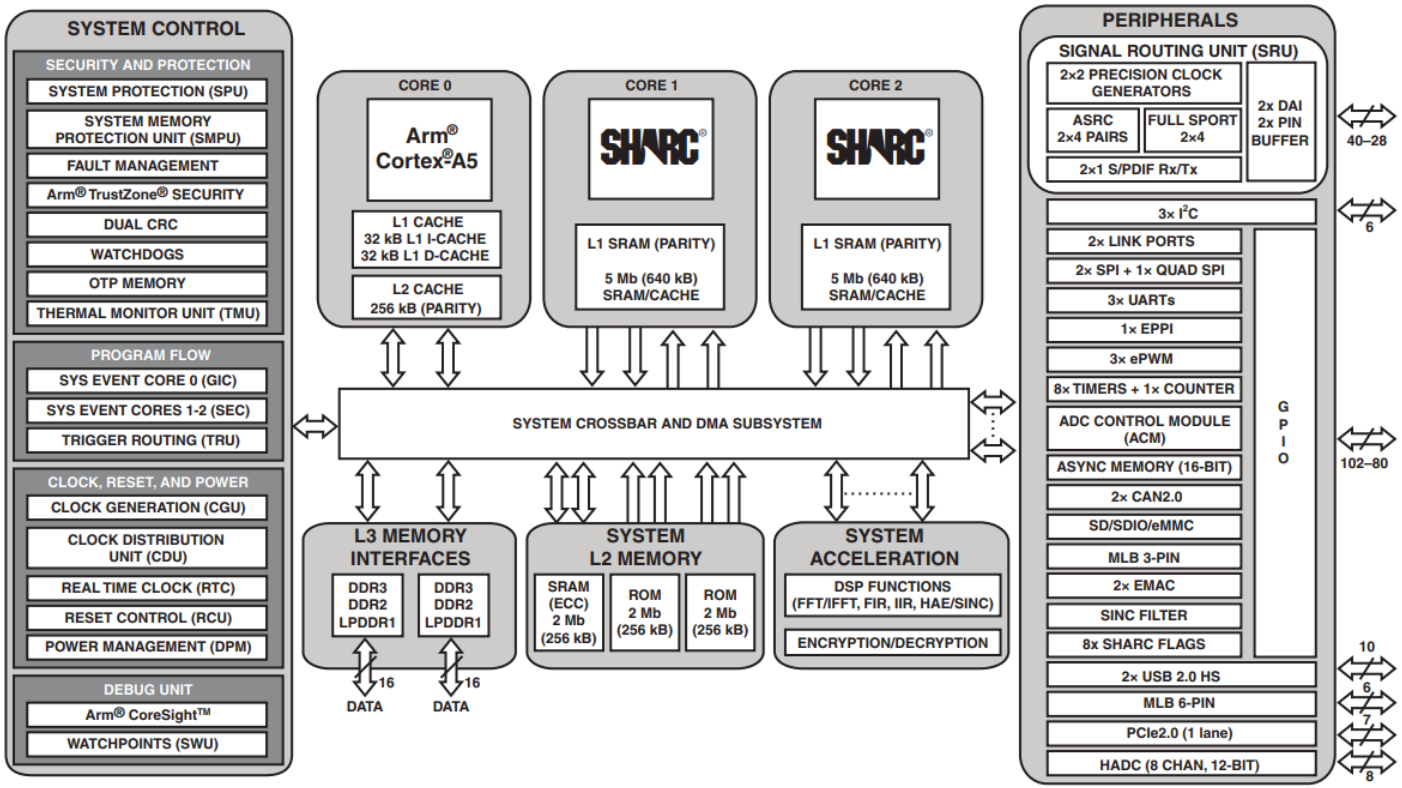
- Speciális, számítást gyorsító HW komponensek és architektúrális megoldások. Például:
  - MAC (szorzás és összeadás párhuzamosan: a+=x*y)
  - Cirkuláris buffer
  - Párhuzamos memóriahozzáférés több memóriablokkhoz
  - Hardveres ciklusszervezés
  - Akár lebegő pontos szorzás egy órajel alatt
  - Osztás, gyökvonás… hardveres támogatása



- Alkalmazási területek:
  - Multimédia: tömörítés, effektek, kódolás (pl. MP3, JPG, MP4), hangszínszabályozás, zajszűrés
  - Szabályozás: motorszabályozás, állapotmegfigyelő, állapotvisszacsatolás
  - Matematikai műveletek: mátrixszorzás, triginometrikus függvények
  - Szenzor jelek mérése: zajszűrés, paraméterbecslés
  - Távközlés: moduláció/demoduláció, kódolás, tömörítés
- Egyre több funkció beépül az általános µC-be is (DSC: digital signal controller)

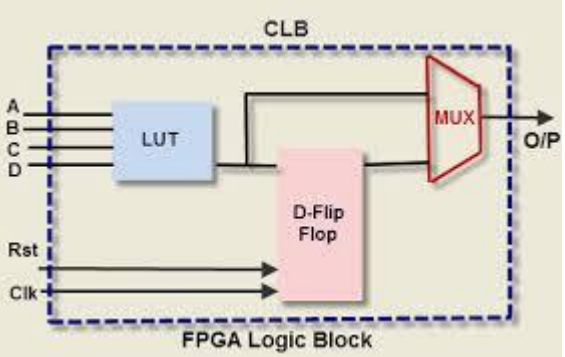
Hibrid megoldások

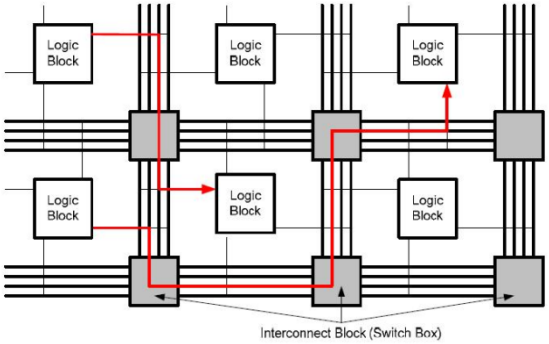
- Analog Devices SC589:
  - 2 db DSP mag: számítási feladatok
  - 1 db ARM cortex-A5: általános feladatok, pl. kommunikáció, perifériák kezelése



FPGA (Field Programmable Gate Array)

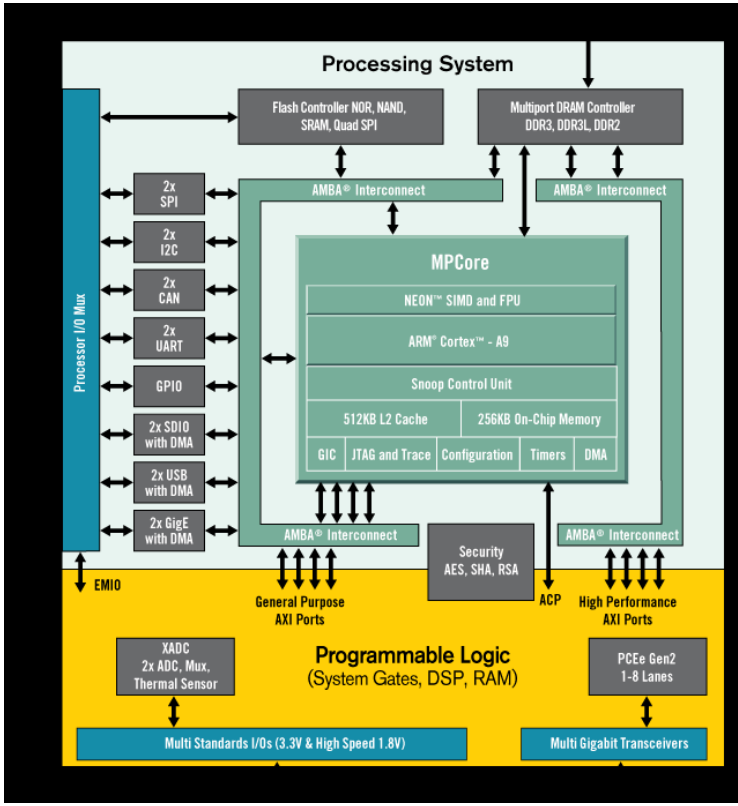
- Általános logikai cellákból (kombinációs logika és FF) felépülő áramkör
- Az általános logikai kapuk közötti kapcsolatok programozhatók
- Flexibilis: nem kell új áramkört tervezni ha módosul a funkcionalitás
- Kiválóan párhuzamosíthatók a feladatok: egy adott egységből (pl. összeadó) „akárhány” példány elhelyezhető
- Nehézkes:
  - A fejlesztés nagy tapasztalatot, speciális tudást igényel
  - Az adott funkcionalitást alacsony szinten (pl. shift-regiszter, multiplexer…) kell megvalósítani, ami időigényes. Manapság már ez is fejlődik, előre definiált modulok…
- Általában nagy számítási igényű, gyorsan és/vagy párhuzamosan végrehajtandó feladatok (gyors mintavételezés, sok bemenő jel, RF és video jelek feldolgozása…)





Hibrid megoldások

- Egy adott alkalmazásban több féle feldolgozó egység használata
- A feladatok dekomponálhatók. Példa:
  - Egy TV készülékben a digitális képfolyam dekódolása FPGA-val vagy DSP-vel,
  - távirányító és menürendszer kezelése mikrokontrollerrel
- Ez nem feltétlenül kell külön IC-n elhelyezkednie (Soc):
  - Léteznek FPGA-ra tölthető ún. soft core processzorok.
  - Léteznek olyan FPGA-k, amelyek egy chipen belül tartalmazzák a uC-t is

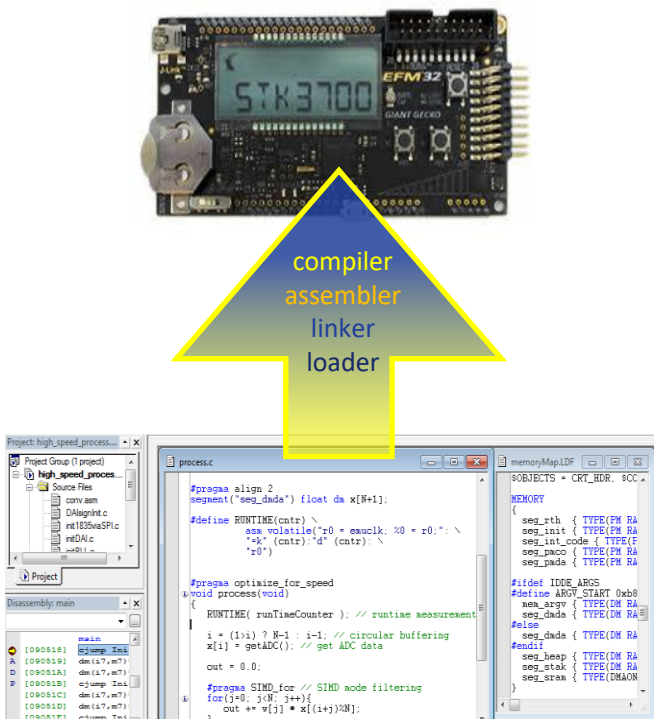


Fordító

A beágyazott szoftverek alacsonyszintű működése
(nézzünk a színfalak mögé)

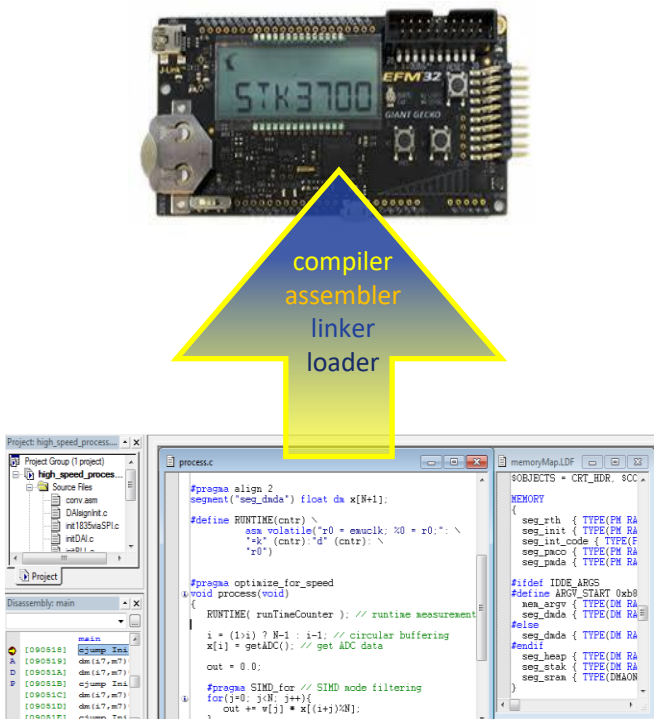
Fejlesztői környezet

- IDE: Integrated Development Environment
- Fejlesztői környezet feladata:
  - Összefogja a rendelkezésre álló programcsomagokat (toolchain), pl.:
    - Fordító: magas szintű kódból assembly szintű kód
    - Assembler: assembly kódból gépi kód
    - Linker: a külön fájlokban található lefordított egységeket egybefűzi
  - Fordítás
  - Debug
  - „Szövegszerkesztés”: a programkód megírásának segítése
    - Pl.: kód kiemelő (code highlighting), automatikus kiegészítés, függvény és változódefiníciók visszakövetése



Fejlesztői környezet

- Fejlesztői környezet feladata:
  - Projektbeállítások kezelése/tárolása
  - Fordító konfigurálása
  - Program letöltése, futtatása
  - Csatlakoztatott beágyazott rendszer kezelése
  - Hibaüzenetek intelligens kezelése
  - HW konfiguráció összeállítása



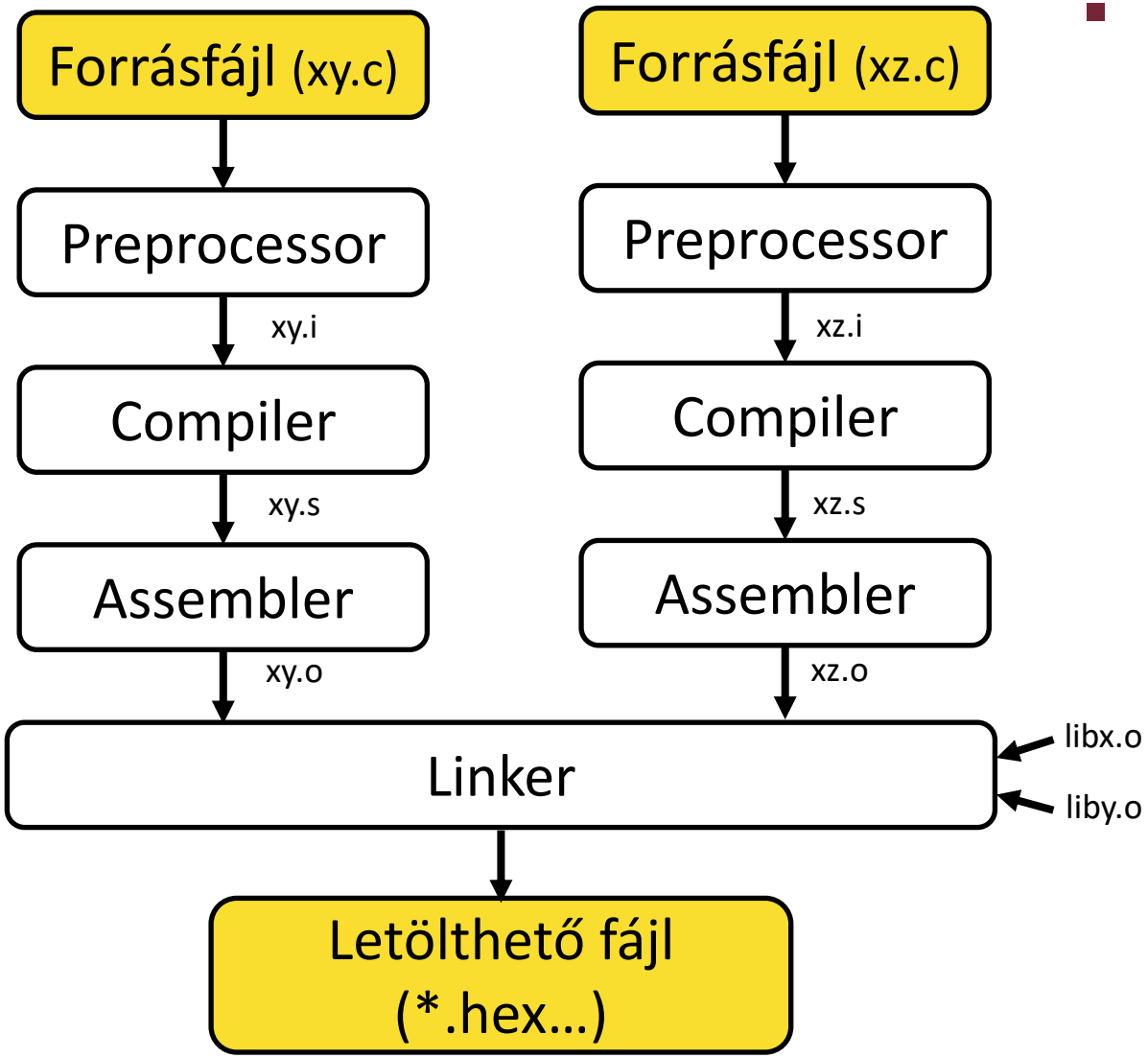
Beágyazott rendszerek fejlesztőkörnyezetei

- Nehéz általános ismertetőt adni, mert a PC-s világtól eltérően rengeteg típusú processzor és platform, ennek megfelelően sok fejlesztői környezet létezik.
  - Speciális tulajdonságok → speciális fordítók
  - Több különböző architektúra, utasításkészlet
- A debugolás nehezebb, mert a processzor egy külön autonóm egység, nem érhető el a PC-ről közvetlenül.
- Fordító és grafikus felület kapcsolata:
  - A fordító (plusz egyéb kiegészítő programok: tool-ok) és a grafikus „kezelői felület” egy egységes rendszert alkot (pl. gyártó szolgáltatja)
  - Általános toolchain (pl. gcc) + editor (pl. Eclipse)

Mintapélda: Simplicity Studio

- Tantárgyban használt fordító: Silicon Laboratories (SiLabs): Simplicity Studio
- Felépítés:
  - Eclipse alapú grafikus környezet
  - gcc alapú fordító
  - A grafikus környezet segít a fordító szolgáltatásainak kezelésében

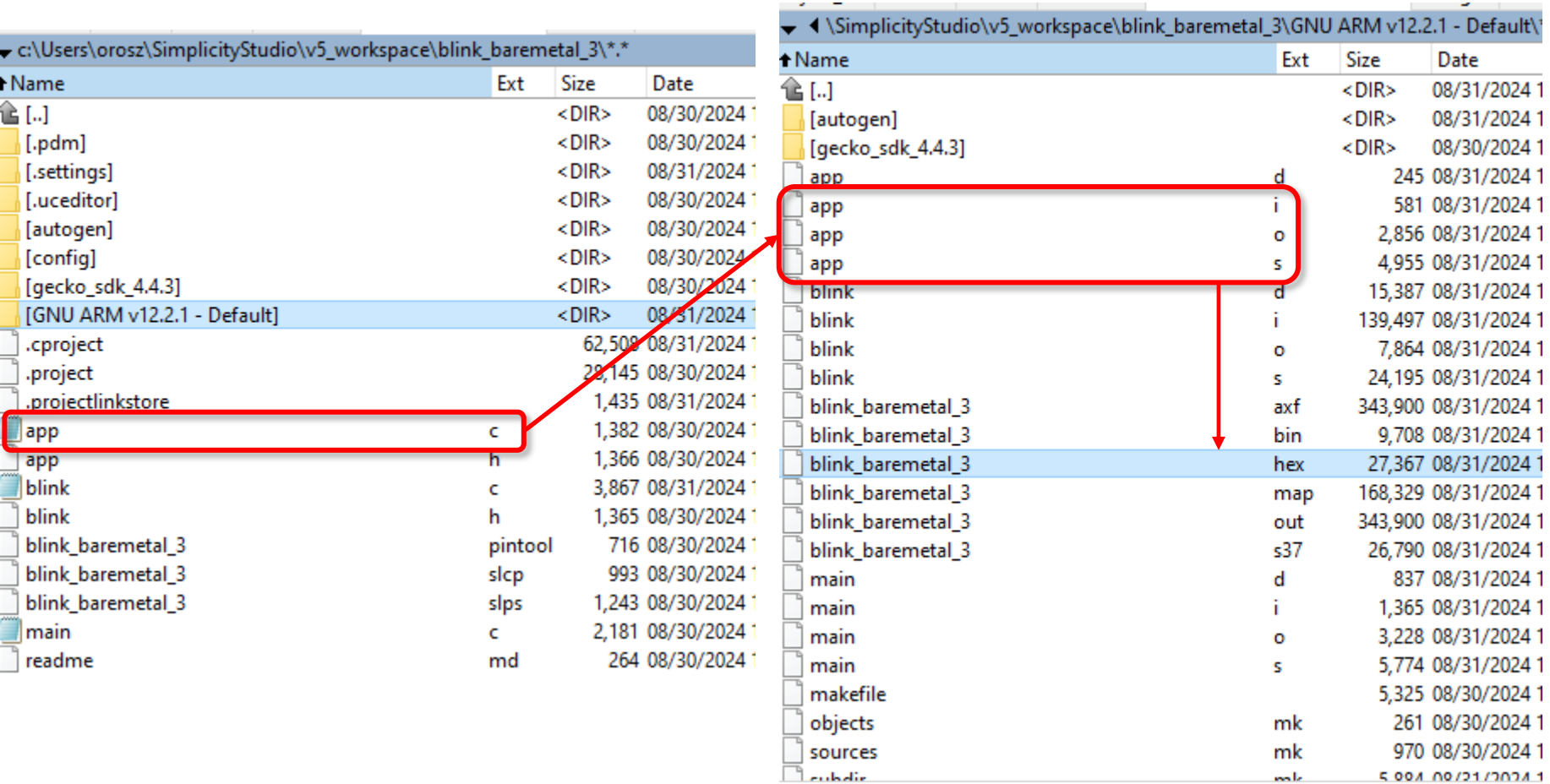
Fordítás menete



- A forrásfájltól a letölthető fájlig több lépésben jutunk el

Fordítás menete

- Mintapélda



Fordítás menete

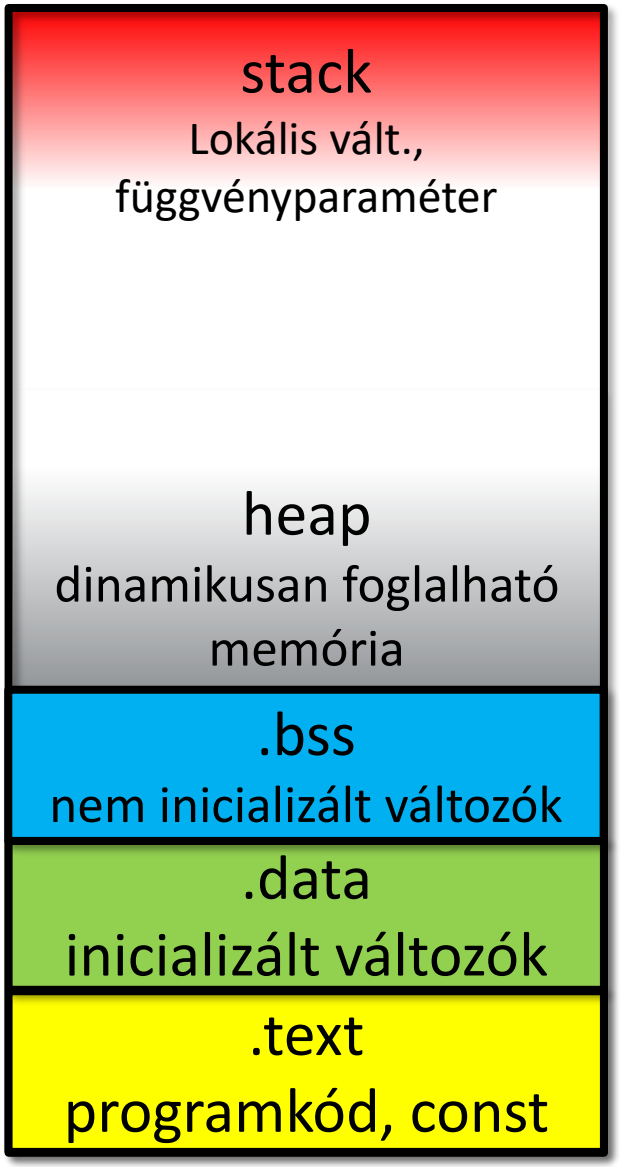
- Preprocessor:
  - Kommentek eltávolítása, header fájlok bemásolása, makrók behelyettesítése
- C nyelvű forráskódok → Assembly kód/object fájl
- Assembly kód → object fájlok
  - object fájl: alapvetően gépi kódra fordított fájl + linker számára szükséges kisegítő információk
- Linker: az object fájlokat integrálja
  - A teljes gépi kódú program előállítása
  - A kiegészítő információk alapján a valódi címek elhelyezése a kódban (pl. különböző C nyelvű fájlokból történő változóhivatkozások vagy függvényhivatkozások feloldása)
  - A változók és függvények elhelyezése a memóriában
    - A linker description fájl tartalmazza az információkat, hogy melyik változó milyen szegmensbe kerül

Fordítás menete

- Kiegészítő információt tartalmazó köztes fájlok (megjeleníthető a -save-temps csatoló segítségével):
  - .i : preprocesszor által feldolgozott fájl (pl. #define-ok, #include-ok behelyettesítve)
  - .s : asm fájl
  - .o : object fájl
  - .d : függőségeket tartalmazó fájl (dependencies, pl. a main.c fájl tartalmazza az init.c-t)
  - .axf: (ARM, máshol .elf néven) debug információkat is tartalmazó object fájl
  - .map: memóriatérkép
- Fordítás végeredménye: eszközre tölthető fájlok (.hex, .bin …)

Szekciók

- Tipikus szekciók:
  - .text → programkód, konstansok
  - .data → inicializált változók
  - .bss → nem inicializált változók
- Fontos, hogy a flash-ben vagy RAM-ban tároljuk
  - Flash általában nagyobb
    - Kisebb a fogyasztása, mint a RAM-nak
    - Programkód tárolása
  - RAM: változók tárolása
- Linker description file
  - Konkrét memóriacímeket definiál a fenti szekciókhoz



Memóriakiosztás

- A linker fájl írja le, hogy melyik szekciók hova kerüljenek a MEM-ban
- Minta linker file (projekt → autogen → linkerfile.ld):

```
MEMORY {
      FLASH (rx) : ORIGIN = 0x0, LENGTH = 0x100000
      RAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 0x20000
 }
```

```
.text : {
  linker_vectors_begin = .;
  KEEP(*(.vectors))
  linker_vectors_end = .;
  __Vectors_End = .;
  __Vectors_Size = __Vectors_End - __Vectors;
  *(.rodata*)
  *(.eh_frame*)
 } > FLASH

 .bss . : {
  . = ALIGN(4);
  __bss_start__ = .; *(SORT_BY_ALIGNMENT(.bss*))
  *(COMMON) . = ALIGN(4);
  __bss_end__ = .;
} > RAM
```

